



MINDSPACE

**Unidade Curricular: Desenvolvimento de Aplicações
Web (DAW)**

**Curso: Licenciatura de Engenharia Informática (LEI)
3º Ano**

Regente da UC: Professor Doutor Rui Paulo

Realizado por: Rodrigo Linhas (a83933)

Ano Letivo: 2025/26

Índice

Resumo.....	3
A Rest API criada (<i>The Rest API created</i>).....	4
Arquitetura de Sistema (<i>Functionality and requirements at client and server sides</i>).....	5
Server Side (Backend).....	5
Client Side (Frontend).....	6
Layout (<i>UX/UI</i>).....	6
Implementação e Instalação (<i>Project Set Up</i>).....	7
Estrutura de Pastas.....	7
<i>Scripts</i> de Execução.....	7
Extras (<i>Three extras/changes</i>).....	8
Gradiente Animado Interativo (<i>Complex CSS/JS System</i>).....	8
Internacionalização Completa (<i>i18n</i>).....	8
Controlo de Acesso Baseado em Funções (<i>RBAC</i>).....	8
REFERÊNCIAS BIBLIOGRÁFICAS.....	9

Resumo

O presente relatório descreve o desenvolvimento da plataforma Mindspace, concebida no âmbito da unidade curricular de Desenvolvimento de Aplicações Web (DAW). O objetivo primordial deste projeto foi a criação de uma plataforma web moderna de "ideia livre", focada na temática da saúde mental, visando proporcionar uma experiência imersiva e relaxante onde o utilizador pode monitorizar o seu bem-estar emocional e receber inspiração diária.

Este documento detalha a solução integrada de uma *API RESTful* desenvolvida em *Node.js/Express*, com persistência de dados em *SQLite* gerida via padrão *Singleton*, e uma interface reativa (*Frontend*) de última geração construída em *React*. Destaca-se a arquitetura de *monorepo* com tipos partilhados, design moderno baseado em *Glassmorphism* com animações complexas e um sistema robusto de internacionalização.

Palavras-chave: *Full Stack, React, Node.js, API REST, SQLite, Glassmorphism, Singleton, TypeScript, Performance.*

A Rest API criada (*The Rest API created*)

A API do Mindspace segue estritamente os princípios *RESTful*, providenciando operações **CRUD** (**C**reate, **R**ead, **U**ppdate, **D**elte) completas através de *endpoints* seguros e validados.

Resumo dos *Endpoints* definidos

Foram definidos os seguintes *endpoints* e *sub endpoints* para o projeto:

Recurso	Métodos	Endpoint	Descrição de Requisitos
Users (<i>api/users</i>)	POST	<i>/signup</i>	Registo de utilizador com validação de esquema (Zod).
	POST	<i>/login</i>	Autenticação (retorna <i>Token JWT</i> + <i>Role</i> associado ao utilizador).
	GET	<i>/</i>	Listagem de utilizadores (somente acessível a <i>admins</i>).
	DELETE	<i>/:id</i>	Remoção de utilizador (somente acessível a <i>admins</i>).
Entries (<i>api/entries</i>)	POST	<i>/</i>	Criação de registo emocional.
	GET	<i>/</i>	Histórico pessoal do utilizador autenticado.
	DELETE	<i>/:id</i>	Remoção de registo (verificação de propriedade).
Quotes (<i>api/quotes</i>)	GET	<i>/</i>	Retorna todas as frases (300 no total e contem suporte a <i>?lang=pt/en</i>).
	GET	<i>/random</i>	Retorna uma frase aleatória (contem suporte a <i>?lang=pt/en</i>).
	GET	<i>?lang=pt/en</i>	Retorna as frases ou em português (pt) ou em inglês (en)

Tabela 1: Todos os *Endpoints* e *Sub Endpoints* disponíveis na API

Arquitetura de Sistema (*Functionality and requirements at client and server sides*)

O sistema opera numa arquitetura *Full Stack* em *monorepo*, partilhando definições de tipos para consistência de dados entre servidor e cliente (e.g. o uso de *Users.ts* tanto no *frontend* como no *backend* está em *shared/types*)

Server Side (Backend)

Desenvolvido em *Node.js* (v18+) com *Express* 4.19, focado em robustez e segurança:

- **Base de Dados: SQLite 5.1.** Gerida através do Padrão de Desenho *Singleton* (ficheiro *db.ts*), para garantir uma instância única de conexão e otimização de recursos;

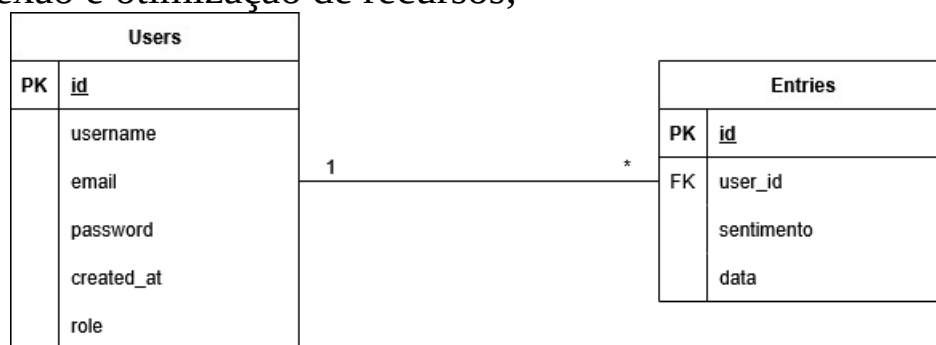


Figura 1: Diagrama Entidade-Relacionamento (ER) da Base de Dados

- **Segurança Avançada:**
 - **Autenticação: Tokens JWT (JSON Web Tokens)** com expiração de 1 hora;
 - **Proteção de Dados: Hashing de palavras-passe via Bcrypt;**
 - **Validação de inputs (Aprendido em Redes 2):** Uso da biblioteca *Zod* para validação estrita em todos os *endpoints*;
 - **Prevenção de brute force (Aprendido em Redes 2):** Implementação de *Helmet* e *Rate Limiting* para mitigar os ataques.

Client Side (Frontend)

A interface é uma *Single Page Application (SPA)* focada na experiência do utilizador:

- **Core:** *React 19.0 com TypeScript 5.5, utilizando Vite* para compilação otimizada.
- **Gestão de Estado:** Utilização da *Context API* para autenticação global (*AuthContext.tsx*) e para *internacionalização* (*LanguageContext.tsx*).
- **Componentes:** Integração de *Chart.js* para visualização de dados e *Twemoji* para renderização consistente de *emojis cross-platform*.

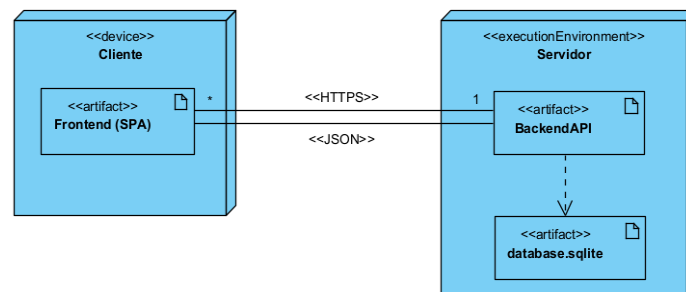


Figura 2: Diagrama de Instalação (Deployment Diagram) associado ao projeto

Layout (UX/UI)

O design visual excede os requisitos base, aplicando conceitos de Interfaces Pessoa-Máquina (**IPM**) para criar uma experiência imersiva e relaxante:

- **Glassmorphism:** Estética de “vidro fosco” conseguida através de fundos semitransparentes (*rgba(255,255,255,0.1)*), desfoque (*backdrop-filter: blur(10px)*) e bordas subtis;
- **Ambiente Vivo:** Fundo animado composto por 5 camadas de gradientes radiais com animações CSS independentes (durações de 20s a 40s), simulando um movimento orgânico;

- **Interatividade:** Integração de *mouse tracking*, onde uma camada de luz segue o cursor do utilizador, aumentando a profundidade visual. Tipografia moderna assegurada pela fonte "Outfit".

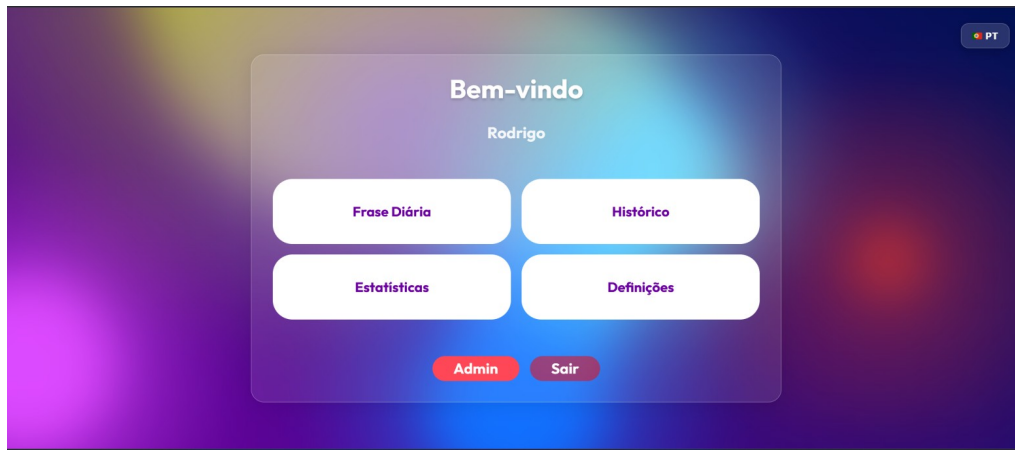


Figura 3: Exemplo demonstrativo da pagina após o login

Implementação e Instalação (*Project Set Up*)

O projeto utiliza uma estrutura de *monorepo* com um *package.json* centralizado na raiz para orquestração eficiente.

Estrutura de Pastas

- *client/*: Código fonte do *Frontend* (React/Vite).
- *server/*: Código fonte do *Backend* (Node/Express).
- *shared/*: Tipos *TypeScript* comuns entre *client/* e *server/*.

Scripts de Execução

A execução é facilitada por *scripts* de automação:

- **Instalação:** *setup.bat* (Windows) e *setup.sh* (Unix/MacOS) para configuração automática de dependências, ou podemos seguir o seguinte fluxo após entrar na *root* do projeto:

```
npm install → npm run build → npm run start
```

- **Desenvolvimento:** *npm run dev* executa cliente e servidor *concurrently* (simultaneamente nas portas 3000 e 5173 respetivamente) otimizando o ciclo de desenvolvimento.
- **Povoamento de dados (Seed):** *npm run populate* reiniciar a base de dados e insere dados de teste (*admins* e *users* com *entries*).

Extras (*Three extras/changes*)

Foram implementadas algumas funcionalidades que representam uma expansão significativa dos requisitos base, mas irei salientar apenas três:

Gradiente Animado Interativo (*Complex CSS/JS System*)

- Sistema complexo que combina animações *CSS keyframes* assíncronas com *listeners JavaScript* para criar fundos reativos ao movimento do rato.
- Valor: Eleva a *UX*, criando imersão alinhada ao tema

Internacionalização Completa (*i18n*)

- Descrição: Suporte bilingue (PT/EN) em tempo real gerido via *LanguageContext.tsx* e os ficheiros *pt.json/en.json*
- Significância: A *i18n* aumenta a acessibilidade e um *market-entry* mais amplo caso seja necessário.

Otimizações de Performance (*Auto Low-Power & GPU*)

- Descrição: Implementação de um sistema inteligente de gestão de recursos, monitorizando os *FPS* em tempo real; se detetar lentidão (*inferior a 30 FPS*) ativa automaticamente um "Modo de Baixo Consumo" que simplifica animações.
- Significância: Garante que a aplicação permanece fluida e acessível.

REFERÊNCIAS BIBLIOGRÁFICAS

- **Recursos Web e Documentação:**

- Baunov G, (s.d.), *Gradients-bg library*, GitHub, Disponível em: <https://github.com/baunov/gradients-bg>
- Chart.js, (s.d.), *Simple yet flexible JavaScript charting*, Disponível em <https://www.chartjs.org>
- Helmet.js, (s.d.), *Secure Express Apps*, Disponível em: <https://helmetjs.github.io>
- Hype4 Academy, (s.d.), *Glassmorphism CSS Generator*, Disponível em: <https://hype4.academy/tools/glassmorphism-generator>
- IETF (Jones, M., et al.), (2015), *JSON Web Token (JWT) - RFC 7519*, Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>
- Krukowski I., (s.d.), *Beginner's guide to internationalization (i18n)*, Lokalise, Disponível em: <https://localise.com/blog/what-is-i18n/>
- React Team, (s.d.), *Documentação Oficial React*, Disponível em: <https://react.dev>
- Vite, (s.d.), *Next Generation Frontend Tooling*, Disponível em: <https://vitejs.dev>
- Zod, (s.d.), *TypeScript-first schema validation*, Disponível em: <https://zod.dev>

- **Material de apoio:**

- Universidade do Algarve, (2025/2026), *Material de Estudo da Unidade Curricular de Análise e Modelação de Sistemas (AMS)*
- Universidade do Algarve, (2025/2026), *Material de Estudo da Unidade Curricular de Desenvolvimento de Aplicações Web (DAW)*
- Universidade do Algarve, (2025/2026), *Material de Estudo da Unidade Curricular de Interfaces Pessoa-Máquina (IPM)*
- Universidade do Algarve, (2025/2026), *Material de Estudo da Unidade Curricular de Redes de Computadores 2 (REDES 2)*
- Universidade do Algarve, (2024/2025), *Material de Estudo da Unidade Curricular de Base de Dados (BD)*